

Sobel Filter on FPGA

Authors:
WhisperingRock

Professor:

Class:
Quarter:
September 2, 2026

Contents

Contents	1
1 Theory of the Sobel Operator	2
1.1 Grayscale Operator	2
1.2 Sobel Operator	2
2 System Architecture	3
3 FPGA IP	4
3.1 Line Buffer	4
3.2 Convolution	6
4 Results	8
5 Analysis	9
6 Conclusion	10
7 Appendix	11
7.1 SystemVerilog	11
7.1.1 Line Buffer	11
7.1.2 Convolution	13
8 References	17

Chapter 1

Theory of the Sobel Operator

1.1 Grayscale Operator

This implementation requires that the RGB image (TODO img type) be flattened into a single intensity value (a grayscale value). This can be done with a few methods.

- Average :

$$p = \frac{R + G + B}{3}$$

- ITU-R B.709 Luminance

$$p = 0.2126R + 0.7152G + 0.0722B$$

I opt'd to use Luminance for its accepted standard.

1.2 Sobel Operator

Uses two 3x3 kernels which are convolved with the original (grayscale) image to calculate approximations of the derivatives. Each kernel, one for horizontal change and one for vertical change, performs a low pass Gaussian filtering $\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$ as well a discrete differentiation $\begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$ on each pixel of the image.

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$

$$\mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Consider the resulting components :

- Magnitude :

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

- Direction of gradient :

$$\Theta = \arctan \left(\frac{\mathbf{G}_y}{\mathbf{G}_x} \right)$$

The results of the operator produce a cheap edge detector that performs some smoothing while retaining a DC gain of 1. For the scope of this project, I'm only using the magnitude.

Chapter 2

System Architecture

Chapter 3

FPGA IP

3.1 Line Buffer

The LineBuffer is designed to ...

- incrementally collect individual input pixel values into the internal storage, using *DIN_AVAIL* and *DIN*, on each clock cycle. *DIN_AVAIL* enables writes while also moving the write pointer to the next position.
- always output a 3 pack of neighbor pixels to *DOUT*, only updating and shifting on a clock'd *DOUT_AVAIL*.
- naturally rollover internal readPtr and writePtr to 512 count (pixels) or reset to zero using *RST*. One might consider the buffer circular using the rollover.

Notes:

- Need at least three writes at or ahead of readPtr before using output data to avoid reading junk.

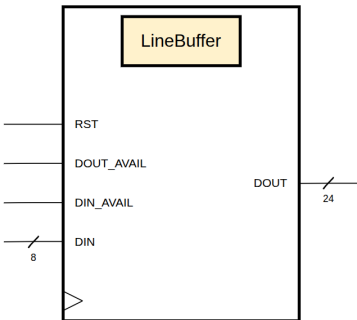


Figure 3.1: LineBuffer Block

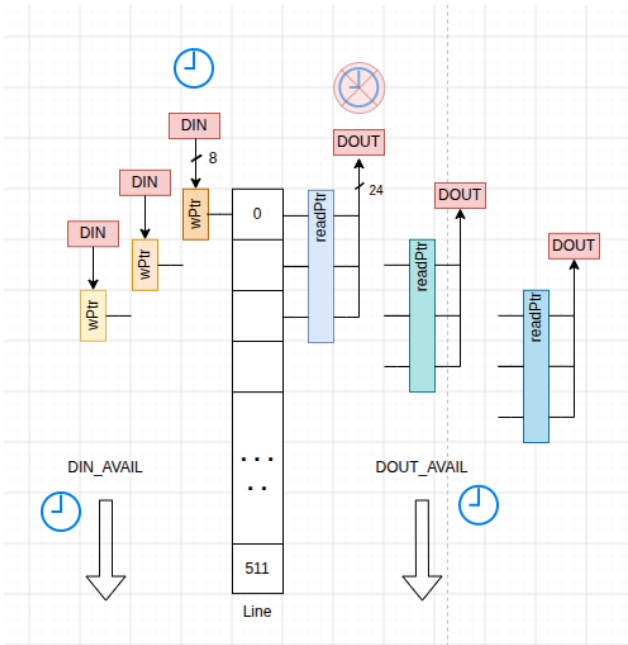


Figure 3.2: LineBuffer Visualized

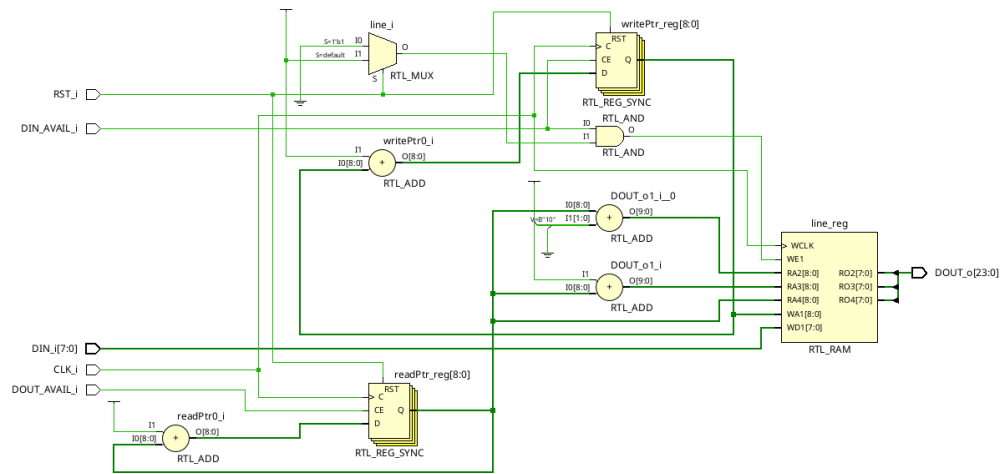


Figure 3.3: LineBuffer RTL

Code Snippets :

- `LineBuffer.sv` [7.1](#)
- `LineBuffer_tb.sv` [7.2](#)

3.2 Convolution

The Convolution module is designed to ...

- Perform a single matrix multiplication as part of convolving the image with the kernel filter.

Always:

$$sum = \sum_{i=0}^8 m[i] * k[i]$$

CLK:

$$Result = \begin{cases} Result & , \text{ if } \overline{MATRIX_VALID} \\ \begin{cases} 0 & , \text{ if } sum < 0 \\ 255 & , \text{ if } sum > 255 \\ \frac{sum}{div} & , \text{ else} \end{cases} & , \text{ if } MATRIX_VALID \end{cases}$$

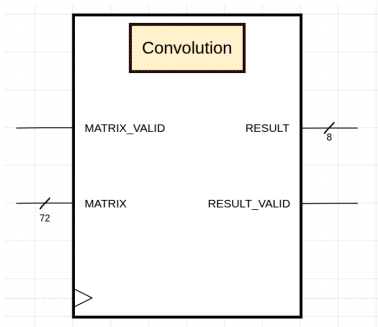


Figure 3.4: Convolution Block

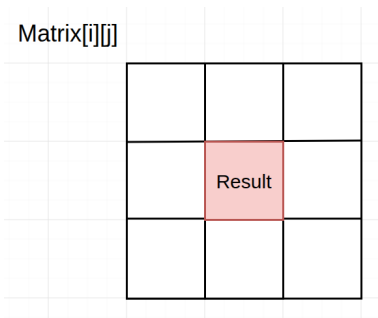


Figure 3.5: Convolution Visualized

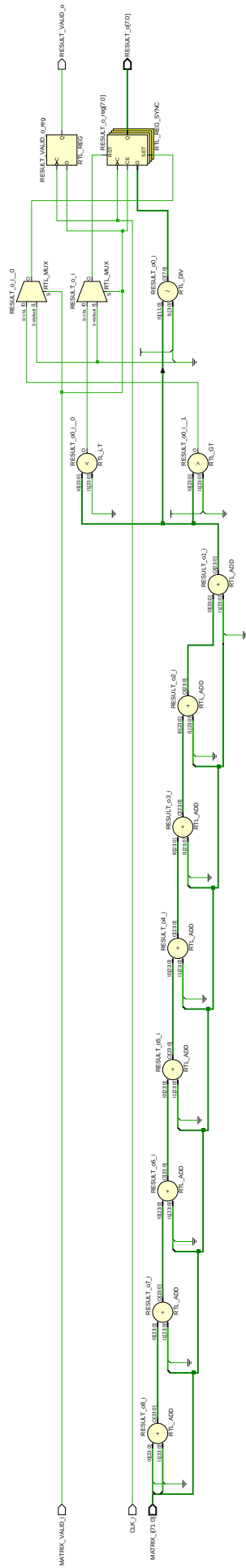


Figure 3.6: Convolution RTL

Code Snippets :

- Convolution.sv [7.3](#)
- Convolution_tb.sv [7.4](#)

Chapter 4

Results

Chapter 5

Analysis

Chapter 6

Conclusion

Chapter 7

Appendix

7.1 SystemVerilog

7.1.1 Line Buffer

Listing 7.1: LineBuffer.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 09:16:29 AM
7  // Design Name:
8  // Module Name: LineBuffer
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:    Gathers a single row (512) of pixel data, one pixel at a time,
13 //                and outputs a single row (3) for a 3x3 kernel. The input_avail
14 //                signal indicates the next pixel is ready to be read and the
15 //                output_avail indicates the downstream module is ready for a
16 //                kernel window slide.
17 //
18 //                How to use :
19 //                - Fill the row largely unrestricted but prevent rollover until ready for new row
20 //                - Only slide kernel row when sobel operation has finished that that position
21 //                - reset can also be used rather than rollover to start a new row/window
22 //
23 // Dependencies:
24 //
25 // Revision:
26 // Revision 0.01 - File Created
27 // Additional Comments:
28 //
29 ///////////////////////////////////////////////////////////////////
30
31
32 module LineBuffer
33 #(parameter int unsigned PWIDTH    = 512,
34   int unsigned PHEIGHT   = 512
35 )
36 (
37   input logic          CLK_i,
38   input logic          RST_i,
39   input logic [7:0]    DIN_i,
40   input logic          DIN_AVAIL_i, // input data ready to be read/sent to this mod
41   input logic          DOUT_AVAIL_i, // output data ready to be read/recieved from this mod
42   output logic [23:0]  DOUT_o,      // packing 3 byte/pixels into a single word. Not using packed
43   //                array to reduce dimension
44 );
45
46 // ~~~~ local params/alloc ~~~~
47 logic [7:0] line [0:PWIDTH-1]; // line contains 512 (bytes) pixels
48 logic [8:0] writePtr;          // 9 bits covers 512 pixel width (2^9)
49 logic [8:0] readPtr;
50
51 // ~~~~ sync logic ~~~~
52 always_ff @(posedge CLK_i)
53 begin
54
55   // ~~ priority 0 : reset r/w ptrs to zero place ~~
56   if(RST_i) begin
57     writePtr    <= 0;
58     readPtr     <= 0;
59   end
60
61   // ~~ priority 1 : read/write ~~
62   else begin
63
64     // ~ Store pixel i in row/line when available ~
65     if(DIN_AVAIL_i == 1'b1) begin
66       line[writePtr] <= DIN_i;
67       writePtr       <= writePtr + 1'b1;
68     end
69
70     // ~ Slide the kernel window (this row in particular) right 1 when prompted ~
71     if(DOUT_AVAIL_i == 1'b1) begin
72       readPtr        <= readPtr + 1'b1;
73     end
74   end
75 end
```

```

75
76     end
77
78     // ~~~~ comb logic ~~~~
79     assign DOUT_o = {line[readPtr], line[readPtr+1], line[readPtr+2]}; // pack 3 pixels for kernel row
80
81 endmodule

```

Listing 7.2: LineBuffer_tb.sv

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 10:33:22 AM
7  // Design Name:
8  // Module Name: LineBuffer_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module LineBuffer_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ local param/allocs ~~~~
29     // ~~ ports ~~
30     logic                clk;
31     logic                reset;
32     logic [7:0]          din;
33     logic                din_avail;
34     logic                dout_avail;
35     logic [23:0]         dout;
36     // ~~ consts ~~
37     localparam int unsigned PWIDTH = 12; // smaller for ease of testing
38     localparam int unsigned PHEIGHT = 12;
39     // ~~ testing ~~
40     testcase tc;
41     logic [31:0]          tnum;
42     bit [11:0][7:0]       arr;
43
44
45     // ~~~~ module instances ~~~~
46     LineBuffer #(PIXELWIDTH(PWIDTH), .PIXELHEIGHT(PHEIGHT)) uut
47     (
48         .CLK_i(clk),                // 1'b
49         .RST_i(reset),              // 1'b
50         .DIN_i(din),                // 8'b
51         .DIN_AVAIL_i(din_avail),    // 1'b
52         .DOUT_AVAIL_i(dout_avail),  // 1'b
53         .DOUT_o(dout)               // 24'b
54     );
55
56     // ~~~~ heartbeat (10ns) ~~~~
57     always begin
58         #5;
59         clk <= !clk;
60     end
61
62     // ~~~~ testing ~~~~
63     initial begin
64
65         // ~~ class instances ~~
66         tc = new();
67
68         // ~~ defaults ~~
69         clk = 1'b0;
70         reset = 1'b1;
71         din = 8'h00;
72         din_avail = 1'b0;
73         dout_avail = 1'b0;
74         #100;
75         reset = 1'b0;
76         #5;
77
78
79         // ~~ TC1 ~~
80         tc.new_test("Filling Line");
81         tnum = tc.get_testnum();
82
83         arr = 96'h12_11_10_09_08_07_06_05_04_03_02_01;
84         din_avail = 1'b1;
85
86         tc.print_subtest("2/3 of writes");
87         din = arr[0];
88         #5;
89         assert(dout == {arr[0], 16'hXX_XX}) else tc.err("DOUT error");
90         #5;
91
92         din = arr[1];
93         #5;
94         assert(dout == {arr[0], arr[1], 8'hXX}) else tc.err("DOUT error");
95         #5;
96
97         tc.print_subtest("Disable write");

```

```

98         din_avail    = 1'b0;
99         din          = arr[2];
100        #5;
101        assert(dout == {arr[0], arr[1], 8'hXX})           else tc.err("DOUT error");
102        #5;
103
104        tc.print_subtest("Re-enable write for 3/3 writes");
105        din_avail    = 1'b1;
106        #5;
107        assert(dout == {arr[0], arr[1], arr[2]})           else tc.err("DOUT error");
108        #5;
109
110        tc.print_subtest("Full buffer prevents new data");
111        din          = arr[3];
112        #5;
113        assert(dout == {arr[0], arr[1], arr[2]})           else tc.err("DOUT error");
114        #5;
115        tc.test_done();
116
117
118        // ~~ TC2 ~~
119        tc.new_test("Reset ptrs");
120        tnum        = tc.get_testnum();
121        reset       = 1'b1;
122        din_avail    = 1'b0;
123        dout_avail   = 1'b0;
124        #10;
125
126        // overwrite first 3 spots with zeros
127        reset       = 1'b0;
128        din_avail    = 1'b1;
129        din          = 8'h00;
130        #30;
131        assert(dout == {24'h00_00_00})                       else tc.err("DOUT error");
132        tc.test_done();
133
134        // ~~ TC3 ~~
135        tc.new_test("Fill to top, then empty");
136        tnum        = tc.get_testnum();
137        // place writePtr back to zero
138        reset       = 1'b1;
139        #10;
140        reset       = 1'b0;
141        din_avail    = 1'b1;
142        dout_avail   = 1'b0;
143
144        // fill
145        din          = arr[0];
146        #10;
147        din          = arr[1];
148        #10;
149        din          = arr[2];
150        #10;
151
152        // fill
153        for(int i = 3; i < 12; i++) begin
154            din      = arr[i];
155            #5;
156            assert(dout == {arr[0], arr[1], arr[2]})           else tc.err("DOUT error");
157            #5;
158        end
159
160        // empty
161        din_avail    = 1'b0;
162        dout_avail   = 1'b1;
163        for(int i = 3; i < 12; i++) begin
164            #5;
165            assert(dout == {arr[i-2], arr[i-1], arr[i]})         else tc.err("DOUT error");
166            #5;
167        end
168
169        #5;
170        assert(dout == {arr[10], arr[11], 8'hXX})             else tc.err("DOUT error");
171        #10;
172        assert(dout == {arr[11], 16'hXX_XX})                   else tc.err("DOUT error");
173        #10;
174        assert(dout == {24'hXX_XX_XX})                         else tc.err("DOUT error");
175        #5;
176
177        tc.test_done();
178
179
180        // ~~ TC4 ~~
181        tc.new_test("R/W synch'd rollover");
182        tnum        = tc.get_testnum();
183
184        //todo
185
186
187        tc.test_done();
188
189
190
191    end
192
193
194 endmodule

```

7.1.2 Convolution

Listing 7.3: Convolution.sv

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //

```

```

6 // Create Date: 08/31/2026 07:34:11 PM
7 // Design Name:
8 // Module Name: Convolution
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //           Input Mat LSB is element zero!!!!
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Convolution
24 #(
25     parameter logic signed [7:0]    KERNEL [0:8] =
26     '{
27         8'd1, 8'd1, 8'd1,
28         8'd1, 8'd1, 8'd1,
29         8'd1, 8'd1, 8'd1
30     },
31
32     parameter logic [15:0]          DIV = 16'd9
33 )
34 (
35     input logic          CLK_i,
36     input logic [71:0]   MATRIX_i,          // 3x3 = 9 pixels, 8 bits each
37     input logic          MATRIX_VALID_i,     // input ready for import
38     output logic [7:0]    RESULT_o,
39     output logic          RESULT_VALID_o     // result ready for export
40 );
41
42 // ~~~~ local param/alloc/const ~~~~
43 logic signed [23:0] sum_comb;
44 localparam signed [8:0] PIXELMIN = 0;
45 localparam signed [8:0] PIXELMAX = 255;
46
47
48 // ~~~~ async logic ~~~~
49 // Note : will perform only once til new data flows in
50 always_comb begin
51
52     // ~~ sanitize between kernel operations ~~
53     sum_comb = '0;
54
55     // ~~ compute an entire kernel sum per CLK ~~
56     for(int i = 0; i < 9; i++) begin
57         sum_comb += KERNEL[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]}); // idx part-selection
58                                     // [start_index +:
59                                     //     width]
60                                     // extending to signed
61     end
62 end
63
64 // ~~~~ sync logic ~~~~
65 always_ff @(posedge CLK_i) begin
66
67     RESULT_VALID_o <= MATRIX_VALID_i; // result likely ready if
68     input is
69
70     if(MATRIX_VALID_i) begin
71         if(sum_comb < PIXELMIN) begin RESULT_o <= 8'd0; end // floor clip
72         else if(sum_comb > PIXELMAX) begin RESULT_o <= PIXELMAX; end // or ceil clip
73         else begin RESULT_o <= sum_comb / DIV; end // or apply gain
74     end
75 end
76
77 endmodule

```

Listing 7.4: Convolution_tb.sv

```

1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 09/01/2026 08:47:08 AM
7 // Design Name:
8 // Module Name: Convolution_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Convolution_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28
29     // ~~~~ local param/allocs ~~~~

```

```

30 // ~~ ports ~~
31 logic                                clk;
32 logic [71:0]                        mat;
33 logic                                mat_valid;
34 logic [7:0]                          result1, result2;
35 logic                                result_valid1, result_valid2;
36 // ~~ consts ~~
37 localparam logic signed [7:0] K_BOX [0:8] =
38 '{
39     1, 1, 1,
40     1, 1, 1,
41     1, 1, 1
42 };
43
44 localparam logic signed [7:0] K_SOBEL_X [0:8] =
45 '{
46     1, 0, -1,
47     2, 0, -2,
48     1, 0, -1
49 };
50
51
52 // ~~ testing ~~
53 testcase tc;
54 logic [31:0]          tnum;
55
56
57 // ~~~~ module instances ~~~~
58 Convolution
59 #(
60     .KERNEL(K_BOX),
61     .DIV(9)
62 )
63 UUT_BOX
64 (
65     .CLK_i(clk),
66     .MATRIX_i(mat),
67     .MATRIX_VALID_i(mat_valid),
68     .RESULT_o(result1),
69     .RESULT_VALID_o(result_valid1)
70 );
71
72 Convolution
73 #(
74     .KERNEL(K_SOBEL_X),
75     .DIV(1)
76 )
77 UUT_SOBEL_X
78 (
79     .CLK_i(clk),
80     .MATRIX_i(mat),
81     .MATRIX_VALID_i(mat_valid),
82     .RESULT_o(result2),
83     .RESULT_VALID_o(result_valid2)
84 );
85
86
87 // ~~~~ heartbeat (10ns) ~~~~
88 always begin
89     #5;
90     clk <= !clk;
91 end
92
93
94 // ~~~~ testing ~~~~
95 initial begin
96
97     // ~~ class instances ~~
98     tc          = new();
99
100    // ~~ defaults ~~
101    clk          = 1'b1;
102    mat          = 72'h00_00_00_00_00_00_00_00_00_00_00_00;
103    mat_valid    = 1'b1;
104    #100;
105
106
107    // ~~ TC1 ~~
108    tc.new_test("1's Mat");
109    tnum          = tc.get_testnum();
110    mat          = 72'h01_01_01_01_01_01_01_01_01_01_01_01;
111    #5;
112    assert(result1 == {8'h01})           else tc.err("BoxBlur result error");
113    assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
114    assert(result2 == {8'h00})           else tc.err("SobelX result error");
115    assert(result_valid1 == {1'b1})       else tc.err("SobelX result_valid error");
116    #5;
117    tc.test_done();
118
119
120    // ~~ TC2 ~~
121    tc.new_test("Floor Mat");
122    tnum          = tc.get_testnum();
123    mat          = 72'h00_00_00_00_00_00_00_00_00_00_00_00;
124    #5;
125    assert(result1 == {8'h00})           else tc.err("BoxBlur result error");
126    assert(result_valid1 == {1'b1})       else tc.err("BoxBlur result_valid error");
127    assert(result2 == {8'h00})           else tc.err("SobelX result error");
128    assert(result_valid2 == {1'b1})       else tc.err("SobelX result_valid error");
129    #5;
130    tc.test_done();
131
132    // ~~ TC3 ~~
133    tc.new_test("5's Mat");
134    tnum          = tc.get_testnum();
135    mat          = 72'h05_05_05_05_05_05_05_05_05_05_05_05;
136    #5;
137    assert(result1 == {8'h05})           else tc.err("BoxBlur result error");

```



```

138         assert(result_valid1 == {1'b1})           else tc.err("BoxBlur result_valid error");
139         assert(result2 == {8'h00})                 else tc.err("SobelX result error");
140         assert(result_valid2 == {1'b1})           else tc.err("SobelX result_valid error");
141         #5;
142     tc.test_done();
143
144     // ~~ TC4 ~~
145     tc.new_test("Ceiling Mat");
146     tnum          = tc.get_testnum();
147     mat           = 72'hFF_FF_FF___FF_FF_FF___FF_FF_FF;
148     #5;
149     assert(result1 == {8'hFF})                     else tc.err("BoxBlur result error");
150     assert(result_valid1 == {1'b1})                 else tc.err("BoxBlur result_valid error");
151     assert(result2 == {8'h00})                     else tc.err("SobelX result error");
152     assert(result_valid2 == {1'b1})                 else tc.err("SobelX result_valid error");
153     #5;
154     tc.test_done();
155
156     // ~~ TC5 ~~
157     tc.new_test("Identity Mat");
158     tnum          = tc.get_testnum();
159     mat           = 72'h01_00_00___00_01_00___00_00_01;
160     #5;
161     assert(result1 == {8'h00})                     else tc.err("BoxBlur result error");
162     assert(result_valid1 == {1'b1})                 else tc.err("BoxBlur result_valid error");
163     assert(result2 == {8'h00})                     else tc.err("SobelX result error");
164     assert(result_valid2 == {1'b1})                 else tc.err("SobelX result_valid error");
165     #5;
166     tc.test_done();
167
168     // ~~ TC6 ~~
169     tc.new_test("3x Identity Mat");
170     tnum          = tc.get_testnum();
171     mat           = 72'h03_00_00___00_03_00___00_00_03;
172     #5;
173     assert(result1 == {8'h01})                     else tc.err("BoxBlur result error");
174     assert(result_valid1 == {1'b1})                 else tc.err("BoxBlur result_valid error");
175     assert(result2 == {8'h00})                     else tc.err("SobelX result error");
176     assert(result_valid2 == {1'b1})                 else tc.err("SobelX result_valid error");
177     #5;
178     tc.test_done();
179
180     // ~~ TC7 ~~
181     tc.new_test("Increasing Mat");
182     tnum          = tc.get_testnum();
183     mat           = 72'h01_02_03___04_05_06___07_08_09; // mat[0] = 09
184     #5;
185     assert(result1 == {8'h05})                     else tc.err("BoxBlur result error");
186     assert(result_valid1 == {1'b1})                 else tc.err("BoxBlur result_valid error");
187     assert(result2 == {8'h08})                     else tc.err("SobelX result error");
188     assert(result_valid2 == {1'b1})                 else tc.err("SobelX result_valid error");
189     #5;
190     tc.test_done();
191
192     // ~~ TC8 ~~
193     tc.new_test("Increasing Mat Reversed");
194     tnum          = tc.get_testnum();
195     mat           = 72'h09_08_07___06_05_04___03_02_01; // mat[0] = 01
196     #5;
197     assert(result1 == {8'h05})                     else tc.err("BoxBlur result error");
198     assert(result_valid1 == {1'b1})                 else tc.err("BoxBlur result_valid error");
199     assert(result2 == {8'h00})                     else tc.err("SobelX result error");
200     // should be -8 but floor clip
201     assert(result_valid2 == {1'b1})                 else tc.err("SobelX result_valid error");
202     #5;
203     tc.test_done();
204
205     // ~~ TC9 ~~
206     tc.new_test("Single-Centered 9");
207     tnum          = tc.get_testnum();
208     mat           = 72'h00_00_00___00_00_09___00_00_00;
209     #5;
210     assert(result1 == {8'h01})                     else tc.err("BoxBlur result error");
211     assert(result_valid1 == {1'b1})                 else tc.err("BoxBlur result_valid error");
212     assert(result2 == {8'd18})                     else tc.err("SobelX result error");
213     assert(result_valid2 == {1'b1})                 else tc.err("SobelX result_valid error");
214     #5;
215     tc.test_done();
216
217     // ~~ TC10 ~~
218     tc.new_test("New data not ready");
219     tnum          = tc.get_testnum();
220     mat           = 72'h00_00_00___00_09_00___00_00_00;
221     mat_valid     = 1'b0;
222     #5;
223     assert(result1 == {8'h01})                     else tc.err("BoxBlur result error");
224     assert(result_valid1 == {1'b0})                 else tc.err("BoxBlur result_valid error");
225     assert(result2 == {8'd18})                     else tc.err("SobelX result error");
226     assert(result_valid2 == {1'b0})                 else tc.err("SobelX result_valid error");
227     #5;
228     tc.test_done();
229 end
endmodule

```

Chapter 8

References

Youtube Inspiration